Comunicación entre procesos

IPC

Señales y Procesos

- *Los procesos, como tareas en ejecución pueden enviar y recibir señales por diversos motivos.*
- *Para responder a estas señales, los procesos acuden a un manejador (signal handler).*
- *Ejemplo: cuando se pulsa <CTRL>C un proceso recibe la señal SIGINT.*

Señales y Procesos

- *El comando “kill” desde el sistema operativo, a pesar de su nombre no necesariamente mata un proceso, sino que le envía una señal.*
- *Un programador puede construir funciones que serán el “handler” de ciertas señales.*
- *Podemos interpretar esto como un proceso de “enmascaramiento”.*

NAME

kill - send a signal to a process

SYNOPSIS

kill [-signal | -s signal] pid ...
kill [-L | -V, --version]
kill -l [signal]

DESCRIPTION

The default signal for kill is TERM. Use -l or -L to list available signals. Particularly useful signals include HUP, INT, KILL, STOP, CONT, and 0. Alternate signals may be specified in three ways: -9 -SIGKILL -KILL. Negative PID values may be used to choose whole process groups; see the PGID column in ps command output. A PID of -1 is special; it indicates all processes except the kill process itself and init.

SIGNALS

The signals listed below may be available for use with kill. When known constant, numbers and default behavior are shown.

Name	Num	Action	Description
0	0	n/a	exit code indicates if a signal may be sent
ALRM	14	exit	
HUP	1	exit	
INT	2	exit	
KILL	9	exit	this signal may not be blocked
PIPE	13	exit	
POLL		exit	
PROF		exit	
TERM	15	exit	
USR1		exit	

Listado de Señales en Unix

```
atsrv:~# kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL
 5) SIGTRAP      6) SIGABRT      7) SIGBUS       8) SIGFPE
 9) SIGKILL     10) SIGUSR1     11) SIGSEGV     12) SIGUSR2
13) SIGPIPE     14) SIGALRM     15) SIGTERM     17) SIGCHLD
18) SIGCONT     19) SIGSTOP     20) SIGTSTP     21) SIGTTIN
22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO
30) SIGPWR      31) SIGSYS      33) SIGRTMIN    34) SIGRTMIN+1
35) SIGRTMIN+2  36) SIGRTMIN+3  37) SIGRTMIN+4  38) SIGRTMIN+5
39) SIGRTMIN+6  40) SIGRTMIN+7  41) SIGRTMIN+8  42) SIGRTMIN+9
43) SIGRTMIN+10 44) SIGRTMIN+11 45) SIGRTMIN+12 46) SIGRTMIN+13
47) SIGRTMIN+14 48) SIGRTMIN+15 49) SIGRTMAX-15 50) SIGRTMAX-14
51) SIGRTMAX-13 52) SIGRTMAX-12 53) SIGRTMAX-11 54) SIGRTMAX-10
55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7  58) SIGRTMAX-6
59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

Signal	Description
SIGABRT	Process abort signal.
SIGALRM	Alarm clock.
SIGFPE	Erroneous arithmetic operation.
SIGHUP	Hangup.
SIGILL	Illegal instruction.
SIGINT	Terminal interrupt signal.
SIGKILL	Kill (cannot be caught or ignored).
SIGPIPE	Write on a pipe with no one to read it.
SIGQUIT	Terminal quit signal.
SIGSEGV	Invalid memory reference.
SIGTERM	Termination signal.
SIGUSR1	User-defined signal 1.
SIGUSR2	User-defined signal 2.
SIGCHLD	Child process terminated or stopped.
SIGCONT	Continue executing, if stopped.
SIGSTOP	Stop executing (cannot be caught or ignored).

Señales y Procesos

- *Por defecto “kill” si no se indica otra cosa envía la señal SIGTERM, que finaliza el proceso, a menos que esta se redirija a una función aportada por el usuario.*
- *La señal “kill -9 (SIGKILL)” es no enmascarable y significa: “muerte segura”.*
- *También existen un par de señales SIGUSR1 y SIGUSR2 que son definibles por el usuario.*

Señales y Procesos

- *Hay una función de biblioteca llamada: kill() definida en signal.h, para enviar una señal desde un proceso a otro.*
- *Se puede levantar una señal dentro del mismo proceso con raise().*
- *Para responder a una señal debemos usar la función signal() y darle un puntero a una función que se convierte en el signal handler.*

Señales y Procesos

- *Forma de utilización de signal:*

```
void (*signal(int sig, void (*func)(int)))(int);
```

- *A signal() hay que pasarle la señal que queremos procesar y un puntero a la función handler.*
- *La función handler que define el usuario debe tener un* **`int`** *como argumento y es de tipo* **`void`**.

Señales y Procesos

- *La llamada a signal() retorna o un error o un puntero a la función que antes oficiaba de handler para que siga manejando las señales que se reciban a futuro.*
- *En síntesis, a signal() le pasamos la señal a manejar, y un puntero a la función handler que tiene un entero como argumento, esta función es de tipo void y devuelve un puntero al handler anterior.*

Uso de función handler

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <signal.h>

void sigint_handler(int sig); /* prototipo */

int main(void) {
  char s[200];

  if (signal(SIGINT, sigint_handler) == SIG_ERR) {
    perror("signal");
    exit(1);
  }

  printf("Ingrese un string: ");
  if (gets(s) == NULL)
    perror("gets");
  else
    printf("Ud. ingresó: \"%s\"\n", s);
  return 0;
}

void sigint_handler(int sig) {
  printf("No me interrupa ahora!\n");
}
```

Resultado

- *Hemos generado un nuevo handler para* **SIGINT** *que evita la interrupción del proceso y escribe un mensaje en pantalla.*

```
atsrv:~# ./sigint
Ingrese un string: No me interrupa ahora!
No me interrupa ahora!
No me interrupa ahora!
No me interrupa ahora!
Ejemplo
Ud. ingresó: "Ejemplo"
atsrv:~#
```

Desde el Sistema Operativo

```
--------------------------
   TERMINAL 1
--------------------------
atsrv:~# ./sigint
Ingrese un string: EjemNo me interrupa ahora!
No me interrupa ahora!
No me interrupa ahora!
Killed
atsrv:~#
```

Proceso a interrumpir

```
--------------------------
   TERMINAL 2
--------------------------
atsrv:~# ps a | grep sigint
  692 pts/0    S+     0:00 ./sigint
atsrv:~# kill -SIGINT 692
atsrv:~# kill -SIGINT 692
atsrv:~# kill -SIGINT 692
atsrv:~# kill -9 692
```

Terminal que interrumpe

fork()

- *fork() es uno de los mecanismos mediante los cuales Unix crea un proceso nuevo a partir de otro.*
- *El proceso hijo obtiene una copia de los datos del padre.*
- *Un proceso padre debería siempre esperar la terminación de sus procesos hijos mediante las funciones:* **`wait()`** *ó* **`waitpid()`**.

fork()

- *Cuando se ejecuta un fork() los resultados pueden ser:*

  0: *Si retorna 0, este es el proceso hijo, se puede obtener el PID del padre con getppid(), el propio se obtiene con getpid().*

  -1: *Si retorna -1 hubo algún problema, el proceso hijo no se creó.*

  Cualquier otro valor retornado por fork() significa que se está tratando con el proceso padre y representa el número de proceso del hijo creado.

Ejemplo de fork()

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

main()
{
  pid_t pid;
  int rv;

  switch(pid=fork()) {
    case -1:
      perror("fork");  /* algo anduvo mal*/
      exit(1);         /* el padre finaliza*/

    case 0:
      printf(" HIJO: Este es el proceso hijo!\n");
      printf(" HIJO: Mi PID es %d\n", getpid());
      printf(" HIJO: El PID de mi padre es %d\n", getppid());
      printf(" HIJO: Ingrese mi valor de retorno (pequeño): \n\n");
      scanf(" %d", &rv);
      printf(" HIJO: Voy a finalizar!\n");
      exit(rv);

    default:
      printf("PADRE: Este es el proceso padre!\n");
      printf("PADRE: Mi PID es %d\n", getpid());
      printf("PADRE: El PID de mi hijo es %d\n", pid);
      printf("PADRE: Espero que mi hijo finalice para ejecutar exit()...\n");
      wait(&rv);
      printf("PADRE: Mi hijo terminó con estado: %d\n", WEXITSTATUS(rv));
                printf("PADRE: Voy a finalizar!\n");
  }
}
```

Resultado de la corrida

```
atsrv:~# ./fork01
PADRE: Este es el proceso padre!
 HIJO: Este es el proceso hijo!
 HIJO: Mi PID es 893
 HIJO: El PID de mi padre es 892
 HIJO: Ingrese mi valor de retorno (pequeño):

PADRE: Mi PID es 892
PADRE: El PID de mi hijo es 893
PADRE: Espero que mi hijo finalice para ejecutar exit()...
18
 HIJO: Voy a finalizar!
PADRE: Mi hijo terminó con estado: 18
PADRE: Voy a finalizar!
atsrv:~#
```

Pipes

- *Pipes (tuberías) es uno de los mecanismos de comunicación entre procesos más antiguos de Unix.*
- *La salida de un proceso, se convierte en la entrada del que sigue y así sucesivamente.*
- *Ej.:*

```
ls -ls | more
resumen | sort | lp
```

Pipes

- *Desde los programas también se pueden generar pipes, el motivo más común es comunicación de un padre con sus hijos.*
- *Para esto se utilizan "file descriptors" como en el trabajo con archivos.*
- *Las funciones fopen(), fclose(), fwrite(), etc. son versiones de alto nivel que usan file descriptors y "system calls" como open(), creat(), close(), write().*

Pipes

- *Cuando arranca un proceso en Unix, tenemos 3 file descriptors por defecto que podemos utilizar:*
  - ***0:*** ***entrada estándar (stdin)***
  - ***1:*** ***salida estándar (stdout)***
  - ***2:*** ***salida de diagnóstico (stderr)***
- *Estos se asocian automáticamente y no hay que abrirlos ni cerrarlos para su utilización.*

Pipes

- *Para abrir una tubería un proceso ejecuta una llamada a pipe(), que retorna un par de file descriptors.*
- *Estos extremos son los que conectamos con otros procesos.*
- *Como los pipes son unidireccionales, en la práctica se cierra la actividad que no se usa en un extremo. P.ej. escritura en un extremo y lectura en el otro.*

Llamada a pipe()

PIPE(2) OpenBSD Programmer's Manual PIPE(2)

NAME

pipe – create descriptor pair for interprocess communication

SYNOPSIS

#include <unistd.h>

int
pipe(int fildes[2]);

DESCRIPTION

The **pipe**() function creates a pipe, which is an object allowing unidirectional data flow, and allocates a pair of file descriptors. The first descriptor connects to the read end of the pipe, and the second connects to the write end, so that data written to fildes[1] appears on (i.e., can be read from) fildes[0]. This allows the output of one program to be sent to another program: the source's standard output is set up to be the write end of the pipe, and the sink's standard input is set up to be the read end of the pipe. The pipe itself persists until all its associated descriptors are closed.

A pipe whose read or write end has been closed is considered widowed. Writing on such a pipe causes the writing process to receive a SIGPIPE signal. Widowing a pipe is the only way to deliver end-of-file to a reader: after the reader consumes any buffered data, reading a widowed pipe returns a zero count.

Pipes

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>

int main()
{
  int pfds[2];
  char buf[30];

  if (pipe(pfds) == -1) {
    perror("pipe");
    exit(1);
  }

  printf("Escribiendo en el file descriptor #%d\n", pfds[1]);
  write(pfds[1], "Esto es una prueba", 19);
  printf("Leyendo desde el file descriptor #%d\n", pfds[0]);
  read(pfds[0], buf, 30);
  printf("Leído: \"%s\"\n", buf);
}
```

```
SALIDA
--------------------------------
Escribiendo en el file descriptor #4
Leyendo desde el file descriptor #3
Leído: "Esto es una prueba"
```

PIPE COMBINADO CON FORK

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    int pfds[2];
    char buf[30];

    pipe(pfds);

    if (!fork())
    {
        printf(" HIJO: escribiendo en el tubo\n");
        write(pfds[1], "test", 5);
        printf(" HIJO: terminando\n");
        exit(0);
    }
    else
    {
        printf("PADRE: leyendo del tubo\n");
        read(pfds[0], buf, 5);
        printf("PADRE: leído \"%s\"\n", buf);
        wait(NULL);
    }
}
```

SALIDA

```
PADRE: leyendo del tubo
 HIJO: escribiendo en el tubo
 HIJO: terminando
PADRE: leído "test"
```

Revisión de pipe()

- *Como ejemplo de utilización de pipe() vamos a generar el código que corresponde a:*

  `ls -l | wc -l`

- *Para esto se cierra la salida del pipe en el padre y se asocia con stdout, también se cierra la entrada en el hijo y se asocia con stdin.*
- *Para las asociaciones se usa: dup()*

NAME

dup, dup2 - duplicate a file descriptor

SYNOPSIS

```
#include <unistd.h>

int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

DESCRIPTION

dup and **dup2** create a copy of the file descriptor *oldfd*.

The old and new descriptors may be used interchangeably. They share locks, file position pointers and flags; for example, if the file position is modified by using **lseek** on one of the descriptors, the position is also changed for the other.

The two descriptors do not share the close-on-exec flag, however.

dup uses the lowest-numbered unused descriptor for the new descriptor.

dup2 makes *newfd* be the copy of *oldfd*, closing *newfd* first if necessary.

Pipes

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
  int pfds[2];

  pipe(pfds);

  if ( fork()) {
    /* Estamos en el proceso PADRE */
    close(1);       /* cierre de la stdout normal */
    dup(pfds[1]);   /* hacemos que stdout sea lo mismo que pfds[1] */
    close(pfds[0]); /* no necesitamos más esto */
    execlp("ls", "ls", NULL);
  }
  else
  {
    /* Estamos en el proceso HIJO */
    close(0);       /* cierre de stdin normal */
    dup(pfds[0]);   /* hacemos que stdin sea lo mismo que pfds[0] */
    close(pfds[1]); /* no necesitamos más esto */
    execlp("wc", "wc", "-l", NULL);
  }
}
```

SALIDA
La cantidad de archivos
en el directorio actual

Named Pipes

- *También se denominan FIFOs.*
- *Son una clase especial de archivos donde los procesos pueden leer y escribir.*
- *Es un pipe verdadero que se ve como si fuera un archivo.*
- *El tubo lo genera un proceso, o el usuario con comandos del sistema operativo.*

Named Pipes

- *Los procesos que se comunican a través del pipe no necesitan tener ninguna relación previa.*
- *Desde el SO se generan así:*

```
atsrv:~# mkfifo --mode=664 tubo
atsrv:~# ls -l tubo
prw-rw-r--  1 root root 0 Mar 24 09:29 tubo
atsrv:~#
```

o *así:*

```
atsrv:~# mknod --mode=664 nuevotubo p
atsrv:~# ls -l nuevotubo
prw-rw-r--  1 root root 0 Mar 24 09:35 nuevotubo
```

Named Pipes

```
MKFIFO(1)                          User Commands                          MKFIFO(1)

NAME
       mkfifo - make FIFOs (named pipes)

SYNOPSIS
       mkfifo [OPTION] NAME...

DESCRIPTION
       Create named pipes (FIFOs) with the given NAMEs.

       Mandatory  arguments  to  long  options are mandatory for short options
       too.

       -m, --mode=MODE
              set permission mode (as in chmod), not a=rw - umask

       --help display this help and exit

       --version
              output version information and exit
```

Named Pipes

```
MKNOD(1)                        User Commands                        MKNOD(1)

NAME
       mknod - make block or character special files

SYNOPSIS
       mknod [OPTION]... NAME TYPE [MAJOR MINOR]

DESCRIPTION
       Create the special file NAME of the given TYPE.
       Mandatory arguments to long options are mandatory for short options too.

       -m, --mode=MODE
              set permission mode (as in chmod), not a=rw - umask

       --help display this help and exit

       --version
              output version information and exit

       Both MAJOR and MINOR must be specified when TYPE is b, c, or u, and they must be omitted
       when TYPE is p.  If MAJOR or MINOR begins with 0x or 0X, it is interpreted as  hexadeci-
       mal; otherwise, if it begins with 0, as octal; otherwise, as decimal.  TYPE may be:

       b      create a block (buffered) special file
       c, u   create a character (unbuffered) special file
       p      create a FIFO
```

Named Pipes

- *Desde un programa se crean con la función:*

```
int mkfifo(const char *pathname, mode_t mode);
```

- *o con la función:*

```
int mknod(const char *pathname, mode_t mode,
dev_t dev);
```

- **mode** *especifica los permisos del pipe, el valor de creación es:*

mode & ~umask

NAME

mkfifo - make a FIFO special file (a named pipe)

SYNOPSIS

```
#include <sys/types.h>
#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);
```

DESCRIPTION

mkfifo makes a FIFO special file with name *pathname*. *mode* specifies the FIFO's permissions. It is modified by the process's **umask** in the usual way: the permissions of the created file are **(***mode* **&** ~**umask)**.

A FIFO special file is similar to a pipe, except that it is created in a different way. Instead of being an anonymous communications channel, a FIFO special file is entered into the file system by calling **mkfifo**.

Once you have created a FIFO special file in this way, any process can open it for reading or writing, in the same way as an ordinary file. However, it has to be open at both ends simultaneously before you can proceed to do any input or output operations on it. Opening a FIFO for reading normally blocks until some other process opens the same FIFO for writing, and vice versa. See **fifo**(4) for non-blocking handling of FIFO special files.

RETURN VALUE

The normal, successful return value from *mkfifo* is **0**. In the case of an error, **-1** is returned (in which case, *errno* is set appropriately).

NAME

mknod - create a special or ordinary file

SYNOPSIS

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int mknod(const char *pathname, mode_t mode, dev_t dev);
```

DESCRIPTION

The system call **mknod** creates a filesystem node (file, device special file or named pipe) named *pathname*, with attributes specified by *mode* and *dev*.

The *mode* argument specifies both the permissions to use and the type of node to be created. It should be a combination (using bitwise OR) of one of the file types listed below and the permissions for the new node.

The permissions are modified by the process's **umask** in the usual way: the permissions of the created node are **(mode & ~umask)**.

The file type must be one of **S_IFREG**, **S_IFCHR**, **S_IFBLK**, **S_IFIFO** or **S_IFSOCK** to specify a normal file (which will be created empty), character special file, block special file, FIFO (named pipe), or Unix domain socket, respectively. (Zero file type is equivalent to type S_IFREG.)

If the file type is **S_IFCHR** or **S_IFBLK** then *dev* specifies the major and minor numbers of the newly created device special file; otherwise it is ignored.

If *pathname* already exists, or is a symbolic link, this call fails with an EEXIST error.

IPCs System V

- *Con el System V fueron introducidas las siguientes formas de comunicación entre procesos:*

  - Colas de mensajes

  - Semáforos

  - Memoria compartida

Identificadores IPC

- *Cada objeto IPC tiene un identificador único asociado con el.*
- *Cuando nos referimos a un objeto IPC estamos hablando de una cola de mensajes simple, un semáforo o un segmento de memoria compartida.*
- *Esto permite al kernel identificar en forma unívoca cada uno de los elementos de IPC.*

Identificadores IPC

- *La generación del identificador se hace con la función:*

```
key_t ftok( char *pathname, char proj );
```

retorna: El nuevo valor de IPC si tuvo éxito
-1 si no tuvo éxito, error en errno.

parámetros:

pathname: un camino, puede ser "."
proj: una letra del abecedario.

Bibliotecas utilizadas:

```
<sys/types.h>
<sys/ipc.h>
```

Identificadores IPC

- *Ejemplo:*

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>

main()
{
  key_t clave;

  clave = ftok( ".", 'a' );
  if( clave == -1 )
    perror( "ftok" );
  else
    printf( "La clave es: %0x\n", clave );
}
```

Salida:

```
La clave es: 6101c9fa
```

Colas de mensajes

- *Pueden ser descriptas como una lista enlazada en el espacio de direcciona-miento del kernel.*
- *Los mensajes son enviados a la cola por un proceso y pueden ser retirados por otro.*
- *Cada mensaje tiene una etiqueta que permite a un programa receptor retirar un mensaje con la etiqueta especificada*

Estructura de un mensaje

- *Es una estructura de C llamada* `msgbuf`

```
struct msgbuf {
   long mtype;     /* tipo de mensaje */
  char mtext[1]; /* texto del mensaje */
};
```

parámetros:

mtype: es un número positivo, etiqueta para cada mensaje en la cola.

mtext: string de modelo, el usuario genera su propia estructura.

- *Hay límites en la cantidad de mensajes, el tamaño de cada mensaje y el tamaño total de la cola.*

Generación de una cola

- *Para generar una nueva cola de mensajes se utiliza:* `msgget`

```
int msgget( key_t key, int msgflg );
```

parámetros:

key: el obtenido con la llamada a ftok()

msgflg: IPC_CREAT ó IPC_EXCL

Con IPC_CREAT se crea la cola si no existía, el valor retornado es la identificación de cola para el usuario (msqid), si existía retorna la identificación que tiene.

IPC_EXCL modifica el comportamiento, si se asocia con IPC_CREAT | IPC_EXCL, retorna -1 si la cola existía.

Generación de una cola

- *Las colas tienen permisos de lectura y escritura como los archivos:*

  Ej.:

```
id = msgget( clave, IPC_CREAT | 0666 );
```

- *De esta forma se indica que la cola tiene permiso de lectura y escritura para todos.*

- *Es buena práctica restringir estos parámetros igual que con los archivos.*

Generación de una cola

- *Los permisos son:*

Bit	Meaning
0400	Read by user.
0200	Write by user.
0040	Read by group.
0020	Write by group.
0004	Read by others.
0002	Write by others.

Envío de mensaje a la cola

- *Una vez generada la cola de mensajes quienes tienen permiso de escritura pueden enviar mensajes con:*

```
int msgsnd( int id, struct msgbuf *msgp,
                    size_t msgsz, int msgflg );
```

retorna: `0 si tuvo éxito`

`-1 si no tuvo éxito, error en errno`

parámetros: **id**: valor retornado por msgget()
msgp: estructura del tipo msgbuf
msgz: tamaño de la estructura
msgflg: indica el modo de ejecución de la sentencia.

Envío de mensaje a la cola

- *Modos de ejecución:*

  0: se usa el comportamiento por defecto, donde el proceso queda bloqueado si la cola está llena y no puede dejar el mensaje.

  IPC_NOWAIT: si la cola de mensajes está llena, el proceso no queda bloqueado esperando el fin de la escritura.

- *Una vez enviado el mensaje a la cola, se puede ver que está allí a nivel del sistema operativo.*

Retiro de mensaje de la cola

- *Quien tiene permiso de lectura puede recuperar un mensaje de la cola con:*

```
ssize_t msgrcv( int id, struct msgbuf *msgp,
      size_t msgsz, long msgtyp, int msgflg );
```

retorna: Nro. de bytes transferidos si tuvo éxito

-1 si no tuvo éxito.

parámetros: **id**: valor retornado por msgget()
msgp: estructura del tipo msgbuf
msgz: tamaño de la estructura
msgtyp: tipo de mensaje (etiqueta)
0 indica primer mensaje.
msgflg: indica el modo de ejecución de la sentencia.

Eliminación de colas

- *Quien tiene permiso de escritura puede eliminar una cola con:*

```
int msgctl ( int msqid, int  cmd,
             struct msqid_ds *buf )
```

parámetros:

msqid: identificador de la cola.
cmd: constante IPC_RMID
msqid_ds: estructura de control.

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x61021a3f 256        antonio    640        40           1
```

Desde el Sistema Operativo

- *En la comunicación entre procesos es posible ver y ejecutar algunas operaciones desde el sistema operativo. Para colas de mensajes tenemos:*

Visualizar: `ipcs -q`

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x61021a3f 256        antonio    640        40           1
```

`ipcs -qu`

```
------ Messages: Status --------
allocated queues = 1
used headers = 1
used space = 40 bytes
```

Desde el Sistema Operativo

Eliminar: `ipcrm msg msqid`

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x61021a3f 256        antonio    640        40           1
```

Visualizar límites: `ipcs -ql`

```
------ Messages: Limits --------
max queues system wide = 128
max size of message (bytes) = 4056
default max size of queue (bytes) = 16384
```

Semáforos

- *Los semáforos sirven para sincronizar operaciones entre procesos en aplicaciones multiproceso.*
- *Esto se da principalmente cuando varios procesos en paralelo tratan de acceder a recursos.*
- *Un semáforo contiene un valor entero y permite o inhibe una operación en función de ese valor.*

Semáforos

- *Esto garantiza que otro proceso no interferirá en el medio de una operación.*
- *Los semáforos se manejan por conjuntos y generalmente cada semáforo de un conjunto tiene un valor tope que indica la cantidad de accesos simultáneos que permite ese recurso.*
- *Cuando una tarea accede a un recurso representado por un semáforo descuenta uno al valor del semáforo.*

Semáforos

- *Cuando esa tarea libera el recurso aumenta uno al valor del semáforo.*
- *Si una tarea trata de decrementar un semáforo que está en 0 queda bloqueada a la espera de la liberación del recurso.*
- *Cuando una de las tareas que lo ocupa lo libera, la que estaba bloqueada podrá continuar y para esto habrá decrementado el semáforo del recurso.*

Ejemplo

- *Si se debe administrar los trabajos de impresión que van a un conjunto de impresoras, podemos utilizar semáforos.*
- *Supongamos que cada impresora tiene un tope posible de trabajos, debemos cargar con ese valor el semáforo de cada una de las impresoras.*
- *Cuando se envía un trabajo a imprimir, el proceso, si puede ingresar, decrementa en uno el semáforo correspondiente.*

Ejemplo

- *Cuando termina la impresión, incrementa en uno el valor del semáforo.*
- *Esto permite acceder a la cola de impresión de esa impresora a otro trabajo que puede haber estado esperando.*
- *En este como en otros casos, si no utilizamos semáforos o mecanismo equivalente, el trabajo que pretende ingresar pertubaría al que se está imprimiendo y/o a los que esperan.*

Inicialización

- *Un conjunto de semáforos se inicializa obteniendo una clave de objeto, igual que con colas de mensajes - ftok() -.*
- *A continuación se llama a la función generadora del conjunto, o asociadora en el caso de que estén creados.*
- *Una vez creado se inicializa cada uno de los semáforos a su valor máximo.*
- *A esto siguen las operaciones de ocupación y liberación de recursos.*

Inicialización

- *Para generar una nuevo conjunto de semáforos se utiliza:* **semget**

```
int semget( key_t key, int nsems, int semflg );
```

parámetros:

key: el obtenido con la llamada a ftok()

nsems: número de semáforos del conjunto.

msgflg: IPC_CREAT ó IPC_CREAT|IPC_EXCL

Con IPC_CREAT se crea el conjunto de semáforos si no existía, el valor retornado es la identificación de conjunto para el usuario (semid), si existía retorna la identificación que tiene.

IPC_EXCL tiene el mismo comportamiento que con las colas de mensajes.

Inicialización

- *La carga de valores en cada semáforo se hace con la función :* **`semctl`**

```
int semctl( int semid, int semnum, int cmd,
                union semun arg );
```

parámetros:

`semid:` el obtenido con la llamada a semget()

`semnum:` indice al semáforo que se controla.

`cmd:` El comando de control (SETVAL, GETVAL, etc)

`semun:` unión de cinco elementos, para nosotros solo tendrá valor el primero que es un entero. Cuando se inicializa tiene el valor máximo del semáforo (SETVAL). Para la lectura tiene el valor cero (GETVAL).

Inicialización

- *parámetros de* **`semctl`** *(cont.)*

```
union semun {
 int val;                /* valor para SETVAL */
 struct semid_ds *buf;   /* buffer para IPC_STAT & IPC_SET */
 ushort *array;          /* arreglo para GETALL & SETALL */
 struct seminfo *__buf; /* buffer para IPC_INFO */
 void *__pad;
};
```

semun: solo nos interesa val, que es el número total de semáforos.
Ejemplo:

```
union semun opciones;
opciones.val = 1;                  // especifica el valor
semctl(id, 0, SETVAL, opciones); // opera en el semáforo de
                                   // índice 0
```

Uso de recurso

- *Esto implica descontar una unidad al semáforo correspondiente al solicitar el uso e incrementar en uno al devolverlo:*

```
int semop( int semid, struct sembuf *sops,
           unsigned nsops );
```

parámetros:

semid: el obtenido con la llamada a semget()

sembuf: estructura para operación.

nsops: número de operaciones en el conjunto.

retorna:

0 si la llamada fue exitosa, -1 en caso contrario.

Estructura sembuf

- *Está declarada en linux/sem.h*

```
struct sembuf {
    ushort sem_num;
    short sem_op;
    short sem_flg;
};
```

campos:

sem_num: El número de semáforo en el conjunto. El primero es 0.

sem_op: La operación que se realiza número positivo, negativo o cero.

sem_flg: bandera que indica el modo de ejecución, con iguales parámetros que en cola de mensajes. (IPC_NOWAIT)

Eliminación de conjuntos

- *Un conjunto de semáforos se elimina con la función de control*

```
int semctl( int semid, int semnum, int cmd,
                union semun arg );
```

parámetros:

semnum: debe ponerse en 0, en este caso no hay semáforo para tratar, sino todo el conjunto.

cmd: Debe ser IPC_RMID.

semun: debe estar en 0.

retorna:

0 si fue exitoso, -1 caso contrario.

Desde el Sistema Operativo

- *Como sucede en todo el mecanismo IPC, es posible ver y ejecutar algunas operaciones sobre los conjuntos de semáforos desde el sistema operativo.*

Visualizar: `ipcs -s`

```
------ Semaphore Arrays --------
key         semid      owner      perms      nsems      status
0x77021a2a 128        antonio    640        5
```

`ipcs -su`

```
------ Semaphore Status --------
used arrays = 1
allocated semaphores = 5
```

Desde el Sistema Operativo

Eliminar: `ipcrm sem semid`

```
------ Semaphore Arrays --------
key         semid      owner      perms      nsems     status
0x77021a2a 128        antonio    640        5
```

Visualizar límites: `ipcs -sl`

```
------ Semaphore Limits --------
max number of arrays = 128
max semaphores per array = 250
max semaphores system wide = 32000
max ops per semop call = 32
semaphore max value = 32767
```

Memoria Compartida

- *Como lo indica su nombre es un área en la que los procesos pueden leer y escribir, después de conectarse al segmento de memoria compartida.*
- *Lo que obtienen los procesos al conectarse es un puntero a memoria.*
- *La función utilizada retorna un identificador del segmento de memoria compartida.*

Memoria Compartida

- *La función utilizada es:*

```
int shmget(key_t key, size_t size, int shmflg);
```

parámetros:

key: el obtenido con la llamada a ftok()

size: tamaño del segmento de memoria reservado.

shmflg: IPC_CREAT ó IPC_CREAT|IPC_EXCL

Con IPC_CREAT se crea el segmento de memoria si no existía, el valor retornado es la identificación del segmento para el usuario (shmid), si existía retorna la identificación que tiene.

IPC_EXCL tiene el mismo comportamiento que con las colas de mensajes.

Memoria Compartida

- *Para conectarse al segmento se utiliza:*

```
void *shmat(int shmid, void *shmaddr, int
                                        shmflg);
```

parámetros:

shmid: valor obtenido en la llamada a shmget()

shmaddr: indica a shmat qué area de memoria se pretende. Esto debería dejarse como decisión al Sistema Operativo, indicando un valor: (void *)0.

shmflg: Si se pretende leer solamente, debería colocarse en este lugar SHM_RDONLY.

Memoria Compartida

- *Para desconectarse del segmento se utiliza:*

```
int shmdt(void *shmaddr);
```

parámetros:

shmaddr: es la dirección obtenida con la función shmat(), que se usó para conectarse al segmento.

- *Al desconectarse del segmento este no se destruye, ni siquiera cuando lo hace el último*

Memoria Compartida

- *Para escribir o leer del área de memoria compartida usamos cualquier función que escriba a partir de un puntero, p. ej.:*

```
strncpy(data, string, size);
```

Siendo:

data: puntero al segmento de memoria compartida
string: la cadena a copiar.
size: tamaño a utilizar dentro de data.

Memoria Compartida

- *Para destruir el área de memoria se utiliza:*

```
shmctl(shmid, IPC_RMID, NULL);
```

parámetros:

shmid: valor obtenido en la llamada a shmget(), utilizado como identificador del segmento.

IPC_RMID: constante que indica remoción.

NULL: esto va en lugar de la estructura de control definida para la función shmctl.

Desde el Sistema Operativo

Eliminar: `ipcrm shm shmid`

```
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x52010015 387        antonio    644        1024       0
```

También se puede utilizar

Eliminar: `ipcrm -m shmid`